

2026 黄金知识汇编（计算机基础与程序设计）

本页由《2026 广东专插本·计算机基础与程序设计·黄金知识汇编》PDF 文字稿整理而成，覆盖 **C语言程序设计**（10 章）与 **数据结构**（8 章）共 **18 章、79 个考点**，去除页码与推广信息，代码已尽量还原为可读 C 代码。

第一部分 C语言程序设计

第一章 程序设计和C语言

考点1. 程序设计语言

一、机器语言

- 1. 一个型号机器语言的指令的集合——机器语言
- 2. 机器语言是由 0 和 1 组成的指令
- 3. 机器语言紧密依赖于计算机的硬件
- 4. 机器语言难学、难记、难写、难修改、难维护
- 5. 在不同计算机之间互不通用

二、高级语言

- 1. 比较接近于人的自然语言（英文）和数学语言
- 2. 高级语言直观易学，易理解，易修改，易维护，易推广，通用性强
- 3. 用高级语言编写的程序，必须先翻译成机器语言程序，此翻译工作由编译系统实现

考点2. 简单C语言程序

注意：\n 换行、%d 格式符号、#include <stdio.h> 标准输入输出函数库。

一、C 程序是由函数构成的

1. C 源程序必须包含一个 `main` 函数
2. 可以包含若干个其他函数
3. 函数是 C 程序的基本单位
4. 被调函数可以是库函数，也可以是用户编制设计的函数
5. 程序全部工作都由各个函数分别完成
6. C语言容易实现程序的模块化

二、一个函数由两个部分组成

1. 函数首部：例如 `int max(int x, int y)`
2. 函数体：
 - 声明部分：定义本函数中所用到的变量以及对本函数所调用函数进行声明
 - 执行部分：由若干个语句组成

三、程序总是从 `main` 函数开始执行

`main` 函数一般放在系统调用库函数后的任意位置。

四、C 程序书写格式自由：一行写多条语句，或者一条语句写在多行。

五、每个语句和数据声明的最后必须有分号：分号是 C 语句的必要组成部分。

六、C语言本身没有输入输出语句：输入和输出操作由库函数 `scanf` 和 `printf` 完成。

七、可以对程序中的任何一行或数行做注释： `//` 单行注释， `/* */` 多行注释。

考点3. 运行C程序的步骤

所谓程序，就是一组计算机能识别和执行的指令。

1. 上机输入和编辑源程序（.c 文件）
2. 对源程序进行编译（.obj 文件）
3. 进行连接处理（.exe 文件）
4. 运行可执行程序，得到运行结果

第二章 数据的存储与运算

考点4. 数据在计算机中是怎样存储的

1、不同进制之间的转换及运算

计算机内部的信息都用二进制表示，分别用 "1" 和 "0" 表示。四种进制：

- **十进制**：由 0-9 这十个数字组成，不能以 0 开头
- **二进制**：由 0 和 1 两个数字组成
- **八进制**：由 0-7 数字组成，为与其他进制区别，开头以 0 开始
- **十六进制**：由 0-9 和 A-F 组成，开头以 0x 开始

(1) 十进制转二进制：除以 2，反向取余数，直到商为 0 终止。例如十进制数 9 对应二进制 1001。

(2) 二进制转十进制：从右向左每位二进制数依次乘以 2 的 0 次方、1 次方.....再累加。例如二进制 1010 对应十进制 10。

(3) 十进制转八进制、十六进制：例如十进制数 1699 对应二进制 011010100011，八进制 3243，十六进制 6A3。

考点5. 整型数据的运算与分析

一、定义变量的一般形式

```
类型名 变量名；
```

例如 `int h, f, x, y;`。变量都必须在使用前定义，指定其类型，即"先定义，后使用"。

二、常量和变量

- 常量：程序运行过程中其值不能改变的量
- 变量：程序运行过程中其值可以改变的量
- 相关概念：变量名、变量地址、存储单元、变量的值

三、变量名（标识符）的取名规则

1. 变量名第一个字符必须是字母或下划线，其后字符必须是字母、数字或下划线。
- 合法： `sum`、`average`、`_total`、`Class`、`day`、`month`、`Student_name`、`tan`
- 不合法： `Zhang-sun`、`Student's`、`263.com`、`$123`、`#33`、`3D64`
1. 大小写字母代表不同的字符，一般变量名用小写字母表示
2. 变量名的长度不是无限的
3. 变量名尽量简单易记、见名知意
4. 同一程序或函数中，不同变量不能取相同名

四、整型常量与整型变量

1. 整型常量：
- 十进制整数，如 `123`、`-456`
- 八进制整数，逢 8 进 1，以 0 开头
- 十六进制整数，逢 16 进 1，用 0~9、a~f 分别代替 0~15，以 0x 开头

1. 整型变量：

类型	关键字
有符号基本整型	<code>[signed] int</code>
无符号基本整型	<code>unsigned int</code>
有符号短整型	<code>[signed] short [int]</code>
无符号短整型	<code>unsigned short [int]</code>

有符号长整型	<code>[signed] long [int]</code>
无符号长整型	<code>unsigned long [int]</code>

考点6. 实型数据的运算与分析

一、实型常量的表示形式（实数在计算机语言中常称为浮点数）

- (1) 十进制小数形式：如 `0.123`、`123.23`、`0.0`
- (2) 指数形式：注意字母 e (E) 之前必须有数字，e (E) 后面的指数必须为整数。如 `123e3` 或 `123E3`

二、实型变量

- (1) 单精度实型变量（`float` 型，4 字节），6-7 位有效位
- (2) 双精度实型变量（`double` 型，8 字节），15-16 位有效位
- (3) 长双精度实型变量（`long double` 型，8/16 字节）

提示： `sizeof(类型名)` 或 `sizeof(变量名)` 测试类型或变量的长度。

考点7. 字符型数据运算

一、字符常量

- (1) 字符常量是用单撇号括起来的一个字符
- (2) 英文字母可以作为字符常量
- (3) 键盘上的字符都可以作为字符常量
- (4) 小写字母和大写字母是不同的字符常量

例如： `'A'`、`'B'`、`'?'`、`'='`

二、转义字符

转义字符必须以反斜杠 `\` 开头。`\` 后只能有一个字符（或代表字符的 8 进制或 16 进制数）。

- `\n` 换行
- `\t` 跳到下一个输出区
- `\0` 常用于字符串中，作为串结束标志

三、字符变量

用来存放字符常量，只能放一个字符。定义形式：

```
char 字符变量列表;  
// 如  
char c1;  
c1 = 'a';
```

字符数据与整型数据在一定条件下通用：`char c='a';` 与 `char c=97;` 等价。

四、字符串常量

是一对双撇号括起来的字符序列。`"How do you do."`、`"CHINA"`、`"a"` 都是合法的字符串。

易错点：`'a'` 是字符常量，`"a"` 是字符串常量，二者含义不同。

考点8. 符号常量

- `#define` 不是 C 语句，行末没有分号
- `#define` 是一个"预编译命令"
- 符号常量一般用大写，以示与变量区别
- 例如：`#define PI 3.1415926`

使用符号常量的好处：含义清楚；需要改变一个常量时能做到"一改全改"；保护所代表的数据不被破坏。

考点9. 算术运算符和算术表达式

一、算术运算符

1. 基本的算术运算符：

- `+` 加法运算符
- `-` 减法运算符
- `*` 乘法运算符
- `/` 除法运算符：两个整数相除的结果为整数，如 `5/3` 结果为 1，舍去小数部分。如果除数或被除数中有一个为负值，舍入方向不固定。VC++ 采取"向零取整"：`5/3=1`、`-5/3=-1`
- `%` 求余运算符（要求两侧均为整数，如 `19%4` 结果为 3）

1. 自增、自减运算符：作用是使变量的值增 1 或减 1

- `++i`、`--i`：在使用 `i` 之前，先使 `i` 的值加（减）1
- `i++`、`i--`：在使用 `i` 之后，使 `i` 的值加（减）1

二、算术表达式

1. 用算术运算符和括号将运算对象（操作数）连接起来的、符合 C 语法规则的式子

2. 运算对象包括常量、变量、函数等

3. 各类数值型数据间的混合运算：

- `char` 和 `short` 型转换为 `int` 型
- `float` 型一律转换为 `double` 型
- 整型（`int`、`short`、`long`）数据与 `double` 型数据进行运算，先将整型转换为 `double` 型
- 注意：字节少的数据转换成字节多的类型

1. 强制类型转换：

- `(double)a`（将 `a` 转换成 `double` 类型）

- (int)(x+y) (将 x+y 的值转换成 int 型)
- (float)(5%3) (将 5%3 的值转换成 float 型)
- 一般形式为：(类型名)(表达式)

考点10. C运算符和C表达式

一、C 运算符

类别	运算符		
算术运算符	+ - * / %		
关系运算符	> < == >= <= !=		
逻辑运算符	! && \	\	`
位运算符	<< >> ~ \	^ &`	
赋值运算符	= 及其扩展赋值运算符		
条件运算符	?:		
逗号运算符	,		
指针运算符	* &		
求字节数运算符	sizeof		
强制类型转换运算符	(类型)		
成员运算符	. ->		
下标运算符	[]		
其他	如函数调用运算符 ()		

二、C 表达式

1. 算术表达式，如 2+6.7*3.5+sin(0.5)
2. 关系表达式，如 x>0、y<z+6

3. 逻辑表达式, 如 `x>0 && y>0` (表示 $x>0$ 与 $y>0$ 同时成立)
4. 赋值表达式, 如 `a=5.6`
5. 逗号表达式, 如 `a=3, y=4, z=8`, 顺序执行, 整个逗号表达式的值是最后一个表达式的值 (值为 8)

三、运算符的优先级与结合性

1. 先按运算符的优先级别高低次序执行
2. 如果优先级别相同, 则按规定的"结合方向"处理:
 - **左结合性**: 运算对象先与左边运算符结合, 即自左至右, 如 `x+y-z`
 - **右结合性**: 运算对象先与右边运算符结合, 即自右向左, 如 `a=b=c=d`

第三章 最简单的C程序—顺序程序设计

考点11. 算法是程序的灵魂

一、什么是算法

一个程序包括以下两方面内容:

1. 对数据的描述 (数据结构): 在程序中指定数据的类型和数据的组织形式
2. 对操作的描述 (算法): 即操作步骤

算法 + 数据结构 = 程序。 算法是灵魂, 数据结构是加工对象, 语言是工具。

二、怎样表示算法

1. 用自然语言表示算法
2. 用流程图表示算法
3. 用 N-S 流程图表示算法
4. 用伪代码表示算法

考点12. 程序的三种基本结构

一、顺序结构：各操作步骤顺序执行，是最简单的一种基本结构。

二、选择结构（又称判断结构或分支结构）：根据是否满足给定条件而从两组操作中选择一种操作。

三、循环结构（又称重复结构）：在一定条件下反复执行某一部分的操作。

考点13. 赋值表达式和赋值语句

一、赋值表达式

- (1) 赋值运算符：`=` 是赋值运算符，作用是将一个数据（或表达式的值）赋给一个变量。如 `a=3;`、`a=b+3;`
- (2) 复合的赋值运算符：在 `=` 之前加上其他运算符可构成复合运算符。`a+=3` 等价于 `a=a+3`；`--`、`*=`、`/=`、`%=` 同理。

二、赋值过程中的类型转换

- 两侧类型一致时，直接赋值
- 两侧类型不一致但都是数值型或字符型，自动将右侧类型转换为左侧类型后赋值

```
- int i; i=3.56; // 结果是 i 的值为 3
- float f; f=23; // 结果是 f 的值为 23.00000
```

三、赋值语句

- 赋值语句由赋值表达式加上一个分号构成，如 `a=3*m;`
 - 赋值表达式末尾没有分号，赋值语句有分号
 - 一个表达式可以包含赋值表达式，但决不能包含赋值语句：
- ```
- if((a=b)>0) t=0; 正确
- if((a=b);>0) t=0; 错误
```

### 四、变量赋初值

可以用赋值语句实现，也可以在定义变量时初始化（后者更方便）：

```
int a, b, c=5;
// 相当于
int a, b, c;
c = 5;
```

## 考点14. 数据输入输出的概念

- 从计算机向输出设备输出数据称为输出，从输入设备向计算机输入数据称为输入
- C语言本身不提供输入输出语句，输入和输出操作由 C 函数库中的函数实现
- `printf`、`scanf`、`putchar`、`getchar`、`puts`、`gets` 不是关键字，而是库函数的名字
- 使用系统库函数时，要在程序中使用预编译命令 `#include`：

- `#include <stdio.h>` （标准方式）

- `#include "stdio.h"`

## 考点15. 字符数据的输入输出

一、用 `putchar` 函数输出一个字符：`putchar(c)`

二、用 `getchar` 函数输入一个字符：`getchar()`

## 考点16. 简单的格式输入与输出

一、用 `printf` 函数输出数据

1. `printf`(格式控制，输出表列)，格式控制包括格式声明和普通字符。例如：`printf("i=%d, c=%c\n", i, c);`
2. 基本格式符号：
  - `d` 格式符：按十进制整型数据的实际长度输出

- `i` 格式符：作用与 `d` 格式符相同
- `c` 格式符：输出一个字符，如 `char ch='a'; printf("%c", ch);`
- `s` 格式符：输出一个字符串，如 `printf("%s", "CHINA");`
- `f` 格式符：输出实数，以小数形式。float 型 6~7 位有效数字，double 型 15~16 位有效数字
- `e` 格式符：以指数形式输出实数

## 二、用 `scanf` 函数输入数据

1. 一般形式：`scanf(格式控制, 地址表列)`，地址表列可以是变量的地址（`&` 取地址符号）或字符串的首地址。例如：`scanf("a=%db=%dc=%d", &a, &b, &c);`

## 考点17. 较复杂的输入输出格式控制

### 1、输出数据时的格式控制

- (1) `%md`：指定输出数据宽度，`m` 为指定宽度。实际位数小于 `m` 则左端补空格，大于 `m` 则按实际位数输出。  
`printf("%4d, %4d", 123, 12345);`
- (2) `%ld`：对于 `int` 型数据占 2 字节的系统，输出长整型数据时在 `d` 前加 `l`
- (3) `%m.nf`：指定输出实数共占 `m` 列，其中 `n` 位小数，数值长度小于 `m` 则左端补空格
- (4) `%-m.nf`：与 `%m.nf` 基本相同，输出数值向左端靠，右端补空格

### 2、输入数据格式控制

以 `%` 开始，以格式字符结束，中间可插入附加字符。可以指定输入数据所占列数，系统自动按它截取所需数据；**输入数据时不能规定精度。**

例如：`scanf("%3d%3d", &a, &b);` 输入 `123456`，系统自动将 123 赋给 `a`，456 赋给 `b`。

## 第四章 选择结构程序设计

## 考点18. 条件判断

一、**条件判断的含义**：用选择结构检查所指定的条件是否满足，并根据判断结果决定执行哪种操作。

### 二、关系运算符及其优先次序（C语言提供 6 种）

- `<` (小于)、`<=` (小于或等于)、`>` (大于)、`>=` (大于或等于) —— 优先级相同(高)
- `==` (等于)、`!=` (不等于) —— 优先级相同(低)

三、**关系、算术、赋值运算符的优先级**：算术运算符(高) > 关系运算符 > 赋值运算符(低)

### 四、关系表达式

- 用关系运算符将两个数值或数值表达式连接起来的式子
- 关系表达式的值是一个逻辑值，即"真"或"假"
- 在 C 的逻辑运算中，以 1 代表"真"，以 0 代表"假"
- 例如  $a=3, b=2, c=1$  时：`a>b` 值为真(1)；`(a>b)==c` 值为真(1)；`b+c<a` 值为假(0)

### 五、逻辑运算符及其优先次序

- 3 种逻辑运算符：`&&` (逻辑与)、`||` (逻辑或)、`!` (逻辑非)
- `&&` 和 `||` 是双目 (元) 运算符，`!` 是单目 (元) 运算符
- 优先次序：`!`  $\rightarrow$  `&&`  $\rightarrow$  `||` (`!` 为三者中最高)

六、**各类运算优先级**（由高到低）：算术运算符 > 关系运算符 > `&&` 和 `||` > 赋值运算符

### 七、逻辑表达式

- 用逻辑运算符将关系表达式或其他逻辑量连接起来的式子
- 逻辑表达式的值是逻辑量"真"或"假"，以 1 代表"真"，以 0 代表"假"
- 将一个非零的数值认作"真"
- 例如判别闰年：能被 4 整除但不能被 100 整除，或能被 400 整除

```
(year%4==0 && year%100!=0) || (year%400==0)
// 值为 1 则闰年，否则非闰年
```

## 考点19. 用if语句实现选择结构

例 4.1 输入两个学生 a 和 b 的成绩，输出其中高的成绩。

程序一（用两个 if）：

```
#include <stdio.h>
void main()
{
 float a, b, max;
 printf("please enter a and b:");
 scanf("%f,%f", &a, &b);
 if (a >= b) max = a;
 if (b > a) max = b;
 printf("max=%6.2f\n", max);
}
```

程序二（用 if-else）：

```
#include <stdio.h>
void main()
{
 float a, b, max;
 printf("please enter a and b:");
 scanf("%f,%f", &a, &b);
 if (a >= b) max = a;
 else max = b;
```

```
printf("max=%6.2f\n", max);
}
```

## if 语句的一般形式

1. if (表达式) 语句
2. if (表达式) 语句1 else 语句2
3. 嵌套形式:

```
if ()
 if () 语句1
 else 语句2
else
 if () 语句3
 else 语句4
```

其中表达式可以是数值表达式、关系表达式或逻辑表达式；语句可以是简单语句也可以是复合语句。

## 考点20. 利用switch语句实现多分支选择结构

学生成绩分类：85 分以上 'A' 等；70~84 分 'B' 等；60~69 分 'C' 等.....

```
switch (grade)
{
 case 'A': printf("85~100\n");
 case 'B': printf("70~84\n");
 case 'C': printf("60~69\n");
 case 'D': printf("<60\n");
 default: printf("error\n");
}
```

## switch 语句的一般形式：

```
switch (表达式)
{
 case 常量表达式1: 语句1
 case 常量表达式2: 语句2
 ...
 case 常量表达式n: 语句n
 default: 语句n+1
}
```

注意：每组语句后面可以加 `break` 语句。

## 说明：

- (1) 括号内"表达式"的值的类型应为整数类型（包括字符型）
- (2) 执行 switch 语句时，先计算表达式值，与各 case 标号比较，相同则转到该 case 后的语句；无匹配则执行 default 后的语句
- (3) 可以没有 default 标号，此时无匹配则不执行任何语句
- (4) 各个 case 标号出现次序不影响执行结果
- (5) 每一个 case 常量必须互不相同，否则互相矛盾
- (6) case 标号只起标记作用，执行完一个 case 后会继续执行下去不再判断，一般用 `break` 跳出 switch 结构；最后一个 case（或 default）子句中可不加 break
- (7) case 子句中包含多个执行语句时可不加花括号，会自动顺序执行
- (8) 多个 case 标号可以共用一组执行语句

## 考点21. 用条件表达式实现简单的选择结构

条件表达式的一般形式：`表达式1 ? 表达式2 : 表达式3`



```
max = (a > b) ? a : b;
// 等价于
if (a > b) max = a;
else max = b;
```

## 第五章 循环结构程序设计

### 考点22. 程序中需要使用循环结构

**一、循环的基本概念：**循环结构又称重复结构，与顺序结构、选择结构是结构化程序设计的三种基本结构，是各种复杂程序的基本构造单元。

**二、构成有效循环需指定两个条件：**

- (1) 需要重复执行的操作，称为循环体
- (2) 循环结束的条件，即在什么情况下停止重复的操作

### 考点23. while和do-while语句实现循环

**一、用 `while` 语句实现循环**

例 5.1 求  $1+2+3+\dots+100$ ：

```
#include <stdio.h>
void main()
{
 int i, sum = 0;
 i = 1;
 while (i <= 100)
```

```

{
 sum = sum + i;
 i++;
}
printf("%d\n", sum);
}

```

`while` 语句一般形式：`while (表达式) 语句`。特点：**先判断条件表达式，后执行循环体语句。**

## 二、用 `do...while` 语句实现循环

求  $1+2+3+\dots+100$ ：

```

#include <stdio.h>
void main()
{
 int i, sum = 0;
 i = 1;
 do
 {
 sum = sum + i;
 i++;
 } while (i <= 100);
 printf("%d\n", sum);
}

```

一般形式：

```

do
 循环体语句
while (表达式);

```

特点：先无条件地执行循环体，然后判断循环条件是否成立。

## 考点24. for语句实现循环

### 一、一般形式和执行过程

求  $1+2+3+\dots+100$ ：

```
for (i = 1; i <= 100; i++)
 sum = sum + i;
```

一般形式：

```
for (表达式1; 表达式2; 表达式3)
 语句
```

特点：

1. for 语句不仅可以用于循环次数已经确定的情况，还可以用于循环次数不确定而只给出循环结束条件的情况
2. for 语句完全可以代替 while 语句

执行过程：

- (1) 先求解表达式1
- (2) 求解表达式2，若其值为真，执行循环体，然后执行第(3)步；若为假，则结束循环，转到第(5)步
- (3) 求解表达式3
- (4) 转回第(2)步继续执行
- (5) 循环结束，执行 for 语句下面的一个语句

## 考点25. 循环嵌套

### 一、循环嵌套概念

1. 一个循环体内又包含另一个完整的循环结构，称为循环的嵌套
2. 内嵌的循环中还可以嵌套循环，即多层循环
3. 3 种循环（while、do...while、for）可以互相嵌套

### 二、循环嵌套实现乘法口诀表：

```
#include "stdio.h"

void main()
{
 int i, j, result;
 for (i = 1; i < 10; i++)
 {
 for (j = 1; j <= i; j++)
 {
 result = i * j;
 printf("%d*%d=%-3d", i, j, result);
 }
 printf("\n");
 }
}
```

## 考点26. 提前结束循环

### 一、用 `break` 语句提前退出循环

`break;` 只能用于循环语句（三种循环语句都可以）和 `switch` 语句之中，不能单独使用。

### 二、用 `continue` 语句提前结束本次循环

`continue`; 作用为结束本次循环，即跳过循环体中下面尚未执行的语句，接着进行下一次是否执行循环的判断。

### 三、`continue` 与 `break` 的区别：

- `continue` 只结束本次循环，而不是终止整个循环的执行
- `break` 结束整个循环过程，不再判断执行循环的条件是否成立

## 考点27. 几种循环比较

- (1) 一般情况下，3 种循环可以互相代替
- (2) 在 `while` 和 `do...while` 循环中，循环体应包含使循环趋于结束的语句
- (3) 用 `while` 和 `do...while` 循环时，循环变量初始化的操作应在 `while` 和 `do...while` 语句之前完成；而 `for` 语句可以在表达式1中实现循环变量的初始化
- (4) 三种循环语句都可以使用 `break` 和 `continue` 语句

## 第六章 利用数组处理批量数据

### 考点28. 为什么要用数组

#### 一、数组的基本概念

1. 数组是一组有序数据的集合。数组中各数据的排列有一定规律，下标代表数据在数组中的序号
2. 用一个数组名和下标唯一确定数组中的元素
3. 数组中的每一个元素都属于同一个数据类型

### 考点29. 怎样定义和引用一维数组

#### 一、定义一维数组：`类型符 数组名[常量表达式];`

注意：常量表达式可以是常量和符号常量，不能包含变量。数组名的命名规则和变量名相同。常量表达式给出元素的个数。下标从 0 开始，如 `int a[10];` 的元素为 `a[0] ~ a[9]`。

## 二、引用一维数组的元素

- 必须先定义数组，才能引用数组中的元素
- 只能逐个引用数组元素，不能一次引用整个数组
- 引用形式：数组名[下标]，如 `a[0]=a[5]+a[2+1]-a[2*3]` 合法；`int n, a[10];` 合法

## 三、一维数组的初始化

### 1. 定义数组时对元素赋初值：

(1) 全部赋值：`int a[10]={0, 1, 2, 3, 4, 5, 6, 7, 8, 9};`

(2) 部分赋值：`int a[10]={0, 1, 2, 3, 4};` 相当于 `int a[10]={0, 1, 2, 3, 4, 0, 0, 0, 0, 0};`

(3) `int a[5]={1, 2, 3, 4, 5};` 可写为 `int a[]={1, 2, 3, 4, 5};`

### 1. 例 6.3 起泡法排序（n 个人按年龄从小到大排列）：

```
int a[10];
int i, j, t;
printf("input 10 numbers :\n");
for (i = 0; i < 10; i++)
 scanf("%d", &a[i]);
printf("\n");
for (j = 0; j < 9; j++)
 for (i = 0; i < 9 - j; i++)
 if (a[i] > a[i+1])
 { t = a[i]; a[i] = a[i+1]; a[i+1] = t; }
printf("the sorted numbers :\n");
for (i = 0; i < 10; i++)
 printf("%d ", a[i]);
printf("\n");
```

## 考点30. 怎样定义和引用二维数组

一、定义二维数组： 类型符 数组名[常量表达式][常量表达式];

二、引用二维数组： 数组名[下标][下标]

- `b[1][2]=a[2][3]/2` 合法
- `int a[3][4]; a[3][4]=3;` 不合法（下标越界）

三、二维数组初始化：

```
int a[3][4]={ {1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12} };
int a[3][4]={1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
int a[3][4]={ {1}, {5}, {9} }; // 等价于 { {1, 0, 0, 0}, {5, 0, 0, 0}, {9, 0, 0, 0} }
int a[3][4]={ {1}, {5, 6} }; // 相当于 { {1}, {5, 6}, {0} }
int a[3][4]={1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12}; // 等价于 int a[][4]={1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
int a[][4]={ {0, 0, 3}, { }, {0, 10} }; // 合法
```

## 考点31. 字符数组

一、定义字符数组及初始化

1. 用来存放字符数据的数组是字符数组
2. 字符数组中一个元素存放一个字符
3. 定义方法与数值型数组类似

```
char c[10];
c[0]='I'; c[1]=' '; c[2]='a'; c[3]='m'; c[4]=' ';
c[5]='h'; c[6]='a'; c[7]='p'; c[8]='p'; c[9]='y';
```

二、字符串和字符串结束标志

- 1. 在 C语言中，是将字符串作为字符数组来处理的
- 2. 关心的是字符串的有效长度而不是字符数组的长度
- 3. C语言规定了字符串结束标志 ' \0 '

```
char c[]={"I am happy"}; // 可写成 char c[]="I am happy";
// 相当于 char c[11]={"I am happy"};
// 相当于 char c[]={ 'I', ' ', 'a', 'm', ' ', 'h', 'a', 'p', 'p', 'y', '\0' }; // 长度11
// 而与下面的不等价（前者长度11，后者长度10）：
// char c[]={ 'I', ' ', 'a', 'm', ' ', 'h', 'a', 'p', 'p', 'y' };

```

三、字符数组的输入输出

- 逐个字符输入输出（ %c ）
- 整个字符串一次输入输出（ %s ）

四、字符串处理函数

| 函数形式                 | 功能                                     |
|----------------------|----------------------------------------|
| gets(字符数组)           | 从终端输入一个字符串到字符数组                        |
| puts(字符数组)           | 将一个字符串（以 ' \0 ' 结束的字符序列）输出到终端          |
| strcat(字符数组1, 字符数组2) | 连接两个字符串，把字符串2接到字符串1后面                  |
| strcpy(字符数组1, 字符串2)  | 将字符串2复制到字符数组1中                         |
| strcmp(字符串1, 字符串2)   | 比较串1和串2。串1=串2返回0；串1>串2返回正整数；串1<串2返回负整数 |
| strlen(字符数组)         | 测试字符串长度                                |
| strlwr(字符串)          | 将字符串中大写字母换成小写字母                        |
| strupr(字符串)          | 将字符串中小写字母换成大写字母                        |



## 第七章 用函数实现模块化程序设计

### 考点32. 函数是什么

**一、函数的基本概念：**函数就是功能（Function）。每一个函数用来实现一个特定的功能，函数的名字应反映其代表的功能。

#### 二、函数说明

1. 一个 C 程序由一个或多个程序模块组成，每一个程序模块作为一个源程序文件
2. 一个源程序文件由一个或多个函数及其他有关内容组成
3. 不论 main 函数出现在什么位置，总是从 main 函数开始执行
4. 所有函数都是平行的，即在定义函数时是分别进行的，互相独立
5. 从用户使用的角度看，函数有两种：库函数、用户自己定义的函数
6. 从函数的形式看，函数分两类：①无参函数 ②有参函数

### 考点33. 函数的定义和调用

#### 一、为什么要定义函数

1. 必须"先定义，后使用"
2. 指定函数名字、函数返回值类型、函数实现的功能以及参数的个数与类型

#### 二、定义函数

1. 无参函数一般形式：

```
类型名 函数名()
{
 函数体
}
```

## 1. 有参函数一般形式：

类型名 函数名(形式参数表列)

```
{
 函数体
}
```

## 三、调用函数

1. 调用无参函数： 函数名() ， 如 `print_star()`

2. 调用有参函数： 函数名(实参表列) ， 如 `max(a, b)`

3. 按函数在程序中出现的位置分 3 种调用方式：

1) **函数语句**：调用没有返回值的函数，单独作为一个语句，如 `print_star();`

2) **函数表达式**：函数出现在一个表达式中，如 `c=2*max(a, b);` 或 `c=max(a, b);`

3) **函数参数**：函数调用作为另一个函数的实参，如 `printf("%d", max(a, b));`

函数举例：

```
#include <stdio.h>

void main()
{
 int max(int x, int y);
 int a, b, c;
 printf("please input two number:");
 scanf("%d,%d", &a, &b);
 c = max(a, b);
 printf("max is %d\n", c);
}

int max(int x, int y)
{
 int z;
```

```
if (x > y) z = x;
else z = y;
return(z);
}
```

#### 四、函数原型和函数声明

1. 在一个函数中调用另一个函数需具备：

- (1) 被调用函数必须是已经定义的函数（库函数或用户自定义函数）
- (2) 如果使用库函数，应在本文件开头加相应的 `#include` 指令
- (3) 如果使用自定义函数，且该函数位置在调用它的函数后面，应进行函数声明

1. 已定义函数的首部再加一个分号，就形成"声明"（函数原型），一般形式有两种：

- `int max(int x, int y);`
- `int max(int, int);`
- 如果被调函数的定义出现在主调函数之前，可以不必加声明
- 原型说明可以放在文件的开头，这时本文件中所有函数都可以使用此函数

### 考点34. 函数的嵌套调用和递归调用

#### 一、函数的嵌套调用

1. 调用一个函数的过程中，又可以调用另一个函数，称为函数的嵌套调用
2. 例 7.3 输入 4 个整数，找出其中最大的数：

```
#include <stdio.h>
void main()
{
 int max_4(int a, int b, int c, int d);
 int a, b, c, d, max;
 printf("4 interger numbers:");
```

```

scanf("%d%d%d%d", &a, &b, &c, &d);
max = max_4(a, b, c, d);
printf("max=%d \n", max);
}
int max_4(int a, int b, int c, int d)
{
 int max(int a, int b);
 int m;
 m = max(a, b);
 m = max(m, c);
 m = max(m, d);
 return(m);
}
int max(int x, int y)
{
 if (x > y) return x;
 else return y;
}

```

## 二、函数的递归调用

1. 在调用一个函数的过程中又出现直接或间接地调用该函数本身，称为函数的递归调用。C语言允许函数的递归调用，分为直接调用本函数、间接调用本函数。

```

int f(int x)
{
 int y, z;
 z = f(y);
 return (2 * z);
}
// 在 f 函数体内又调用了 f 函数

```

1. 例 7.6 有 5 个学生坐在一起，第  $n$  个比第  $n-1$  个大 2 岁，第 1 个 10 岁，求第 5 个学生多少岁：

```

#include <stdio.h>
int age(int n)
{
 int c;
 if (n == 1) c = 10;
 else c = age(n - 1) + 2;
 return(c);
}
void main()
{
 printf("%d\n", age(5));
}

```

1. 例 7.5 分别用递推方法和递归方法求  $n!$ , 即  $1 \times 2 \times \dots \times n$ :

```

#include <stdio.h>
void main()
{
 long fac(int n);
 int n, y;
 printf("input an integer number:");
 scanf("%d", &n);
 y = fac(n);
 printf("%d!=%ld\n", n, y);
}
long fac(int n)
{
 long f;
 if (n < 0) printf("n<0, data error!");
 else if (n == 0 || n == 1) f = 1;
 else f = fac(n - 1) * n;
 return(f);
}

```

## 考点35. 数组作为函数参数

一、用数组元素作函数实参：与变量作为实参一样，传递方式是单向值传递。

### 二、用数组名作函数参数

- 希望在函数中处理整个数组的元素时，可以用数组名作为函数实参
- 此时只是将数组的首元素的地址传递给所对应的形参，对应的形参应当是数组名或指针变量
- 例 7.9 用一个函数实现用选择法对 10 个整数按升序排列：

```
#include <stdio.h>
void main()
{
 void sort(int array[], int n);
 int a[10], i;
 printf("enter the array:\n");
 for (i = 0; i < 10; i++) scanf("%d", &a[i]);
 sort(a, 10);
 printf("The sorted array:\n");
 for (i = 0; i < 10; i++) printf("%d ", a[i]);
 printf("\n");
}

void sort(int array[], int n)
{
 int i, j, k, t;
 for (i = 0; i < n - 1; i++)
 {
 k = i;
 for (j = i + 1; j < n; j++)
 if (array[j] < array[k]) k = j;
 t = array[k];
 array[k] = array[i];
 array[i] = t;
 }
}
```

## 考点36. 变量的作用域和生存期

### 一、变量的作用域——局部变量和全局变量

1. **局部变量**：在函数和复合语句内定义的变量（内部变量），只在本函数或复合语句内有效
2. **全局变量**：在函数之外定义的变量（外部变量/全程变量），有效范围为从定义位置到本源文件结束，可被本文件所有函数共用。若同一源文件中外部变量与局部变量同名，则在局部变量作用范围内外部变量被"屏蔽"

### 二、变量的存储方式和生存期

- 变量的生存期：变量值存在的时间
- 两种存储方式：静态存储方式（程序运行期间由系统分配固定存储空间）、动态存储方式（根据需要动态分配存储空间）
- 全局变量采用静态存储方式；函数中定义的变量在函数调用时分配动态存储空间

```
#include <stdio.h>
void main()
{
 int fac(int n);
 int i;
 for (i = 1; i <= 5; i++)
 printf("%d!=%d\n", i, fac(i));
}
int fac(int n)
{
 static int f = 1;
 f = f * n;
 return(f);
}
```

### 三、作用域和生存期小结

1. 对一个变量的属性可以从两方面分析：作用域（空间）和生存期（时间）

2. 二者有联系但不是同一回事

## 考点37. 内部函数和外部函数

### 一、内部函数

- 只能被本文件中其他函数所调用的函数
- 定义时在函数名和函数类型前面加 `static` : `static` 类型标识符 函数名 (形参表)
- 内部函数又称静态函数, 用 `static` 声明
- 通常把只能由本文件使用的函数和外部变量放在文件开头, 前面冠以 `static` 使之局部化, 其他文件不能引用
- 提高了程序的可靠性

### 二、外部函数

- 定义函数时在函数首部最左端加关键字 `extern`, 则此函数是外部函数, 可供其他文件调用
- 如 `extern int fun(int a, int b)`
- 如果在定义函数时省略 `extern`, 则默认为外部函数

## 第八章 善于使用指针

### 考点38. 什么是指针

#### 一、指针与指针变量的基本概念

在 C 语言中, 将地址形象化地称为"指针", 意思是通过它能找到以它为地址的内存单元。

```
int a;
a_pointer = &a;
```

- 一个变量的地址称为该变量的"指针" (例如地址 2000 是变量 a 的指针)
- 如果有一个变量专门用来存放另一变量的地址 (即指针), 则它称为"指针变量"



- `a_pointer` 就是一个指针变量，其值是地址（即指针）
- **"指针"和"指针变量"是不同的概念**：指针是一个地址，而指针变量是存放地址的变量。可以说变量 `a` 的指针是 2000，而不能说 `a` 的指针变量是 2000

## 考点39. 指针变量

### 一、使用指针变量访问变量：

```
#include <stdio.h>

void main()
{
 int a, b;
 int *pointer_1, *pointer_2;
 a = 100; b = 10;
 pointer_1 = &a;
 pointer_2 = &b;
 printf("a=%d, b=%d\n", a, b);
 printf("*pointer_1=%d, *pointer_2=%d\n", *pointer_1, *pointer_2);
}
```

### 二、定义指针变量：基类型 \*指针变量名；，如 `int *pointer_1, *pointer_2;`

- `int` 是为指针变量指定的"基类型"，指定指针变量可指向的变量类型
- 如 `pointer_1` 可以指向整型变量，但不能指向浮点型变量

### 三、引用指针变量

1. 给指针变量赋值： `p = &a;`
2. 引用指针变量指向的变量： `p = &a; *p = 1;` 则 `printf("%d", *p);` 输出 1
3. 引用指针变量的值： `printf("%o", p);`

两个有关的运算符：

- `&` 取地址运算符, `&a` 是变量 `a` 的地址
- `*` 指针运算符 ("间接访问"运算符), 如果 `p` 指向变量 `a`, 则 `*p` 就代表 `a`; `k=*p;` (把 `a` 的值赋给 `k`); `*p=1;` (把 1 赋给 `a`)

## 四、指针变量作为函数参数

例 8.3 对输入的两个整数按大小顺序输出, 用指针变量作函数参数:

```
#include <stdio.h>

void main()
{
 void swap(int *p1, int *p2);
 int a, b;
 int *pointer_1, *pointer_2;
 scanf("%d,%d", &a, &b);
 pointer_1 = &a;
 pointer_2 = &b;
 if (a < b) swap(pointer_1, pointer_2);
 printf("max=%d, min=%d\n", a, b);
}

void swap(int *p1, int *p2)
{
 int temp;
 temp = *p1;
 *p1 = *p2;
 *p2 = temp;
}
```

## 考点40. 通过指针引用数组

### 一、数组元素的指针

1. 一个变量有地址, 一个数组包含若干元素, 每个数组元素都有相应的地址
2. 指针变量可以指向数组元素 (把某一元素的地址放到一个指针变量中)

3. 所谓数组元素的指针就是数组元素的地址

4. 可以用一个指针变量指向一个数组元素：

```
int a[10] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};
int *p;
p = &a[0]; // 等价于 p = a;
```

注意：数组名 `a` 不代表整个数组，只代表数组首元素的地址。`p = a;` 的作用是"把 `a` 数组的首元素的地址赋给指针变量 `p`"，而不是"把数组 `a` 各元素的值赋给 `p`"。

## 二、通过指针引用数组元素

1. 引用数组元素可用下面两种方法：

(1) 下标法，用数组名加下标如 `a[i]`

(2) 指针法（地址法），`*(a+i)` 或 `*(p+i)`（其中初值 `p=a`）

1. 指针运算：

(1) 如果指针变量 `p` 已指向数组中的一个元素，则 `p+1` 指向同一数组中的下一个元素，`p-1` 指向同一数组中的上一个元素。如 `float a[10], *p=a;`，设 `a[0]` 地址为 2000，则 `p` 的值为 2000，`p+1` 的值为 2004

(2) 如果 `p` 的初值为 `&a[0]`，则 `p+i` 和 `a+i` 就是数组元素 `a[i]` 的地址

(3) `*(p+i)` 或 `*(a+i)` 是 `p+i` 或 `a+i` 所指向的数组元素，即 `a[i]`

(4) 如果 `p` 指向 `a[0]`，`p++` 后 `p` 的值改变，指向下一个数组元素 `a[1]`

(5) 如果指针 `p1` 和 `p2` 都指向同一数组，可以执行 `p2-p1`，结果是两个地址之差除以数组的长度，表示 `p2` 所指元素与 `p1` 所指元素之间差几个元素

1. 例 8.7 通过指针变量读入数组的 10 个元素，然后输出：

```
#include <stdio.h>
void main()
{
```

```

int *p, i, a[10];
p = a;
for (i = 0; i < 10; i++) scanf("%d", p++);
p = a; // 使指针 p 重新指向 &a[0]
for (i = 0; i < 10; i++, p++)
 printf("%d ", *p);
printf("\n");
}

```

### 三、用数组名作函数参数

- 用数组名作函数参数时，因为实参数组名代表该数组首元素的地址，形参应该是一个指针变量
- C 编译都是将形参数组名作为指针变量来处理的
- 实参数组名是指针常量，但形参数组名是按指针变量处理
- 例 8.9 编写一个函数用选择法对 10 个整数按由大到小顺序排序，用数组名作实参：

```

#include <stdio.h>
void main()
{
 void sort(int x[], int n);
 int *p, i, a[10];
 p = a;
 for (i = 0; i < 10; i++) scanf("%d", p++);
 p = a;
 sort(p, 10);
 for (p = a, i = 0; i < 10; i++)
 {
 printf("%d ", *p);
 p++;
 }
 printf("\n");
}
void sort(int x[], int n)
{
 int i, j, k, t;
 for (i = 0; i < n - 1; i++)

```

```

{
 k = i;
 for (j = i + 1; j < n; j++)
 if (x[j] > x[k]) k = j;
 if (k != i)
 { t = x[i]; x[i] = x[k]; x[k] = t; }
}
}

```

## 考点41. 通过指针引用字符串

### 一、字符串的表示形式

1. 可以用两种方法访问一个字符串：

- (1) 用字符数组存放一个字符串，用字符数组名和下标访问元素，也可通过字符数组名用 `%s` 格式符输出
- (2) 用字符指针指向一个字符串（不定义字符数组，而定义一个字符指针）

1. 例 8.10 定义字符指针，使它指向一个字符串：

```

#include <stdio.h>
void main()
{
 char *string = "I love China!";
 printf("%s\n", string);
}

```

### 二、字符指针作函数参数

- 如果想把一个字符串从一个函数"传递"到另一个函数，可以用地址传递的办法，即用字符数组名作参数，也可以用字符指针变量作参数
- 在被调用的函数中可以改变字符串的内容；在主调函数中可以引用改变后的字符串
- 例 8.13 复制字符串，用函数调用实现：

```

#include <stdio.h>

void main()
{
 void copy_string(char *from, char *to);
 char *a = "I am a teacher.";
 char b[] = "You are a student.";
 char *p = b;
 printf("a=%s\n b=%s\n", a, p);
 printf("copy string a to string b:\n");
 copy_string(a, p);
 printf("a=%s\n b=%s\n", a, p);
}

void copy_string(char *from, char *to)
{
 for (; *from != '\0'; from++, to++)
 { *to = *from; }
 *to = '\0';
}

```

调用函数时实参与形参的对应关系（实参 / 形参）：字符数组名/字符数组名、字符数组名/字符指针变量、字符指针变量/字符指针变量、字符指针变量/字符数组名。

### 三、字符指针变量和字符数组的区别

1. 字符数组由若干个元素组成，每个元素中放一个字符，而字符指针变量中存放的是地址（字符串第 1 个字符的地址）
2. 赋值方式不同
3. 字符数组在编译时为它分配内存单元，有确定的地址；字符指针变量分配内存单元存放一个字符变量的地址
4. 指针变量的值是可以改变的
5. 对字符数组可以用下标法和地址法 `*(a+5)` 引用数组元素；如果字符指针变量 `p=a`，则也可以用 `*(p+5)`
6. 字符数组中各元素的值是可以改变的（可再赋值），但字符指针变量指向的字符串常量中的内容不可被取代（不能对它们再赋值）

## 第九章 使用结构体类型处理组合数据

## 考点42. 定义和使用结构体变量

### 一、建立结构体类型变量

- (1) 用户自己建立由不同类型数据组成的组合型的数据结构，称为结构体
- (2) 声明一个结构体类型的一般形式为：

```
struct 结构体名
{
 成员表列
}; // 成员声明形式：类型名 成员名；
```

- (3) 结构体类型示例：

```
struct Student
{
 int num;
 char name[20];
 char sex;
 int age;
 float score;
 char addr[30];
};
```

### 二、结构体类型变量

- 前面只是建立了一个结构体类型，相当于一个模型，并没有定义变量，其中并无具体数据，系统也不分配存储单元
1. 先声明结构体类型，再定义该类型变量：`struct student student1, student2;`
  2. 在声明类型的同时定义变量：

```
struct student
{
 int num;
 char name[20];
 char sex;
 int age;
 float score;
 char addr[30];
} student1, student2;
```

1. 不指定类型名而直接定义结构体类型变量：`struct { 成员表列 } 变量名表列;`（指定了一个无名的结构体类型）

#### 说明：

- (1) 结构体类型与结构体变量是不同的概念，只能对变量赋值、存取或运算，编译时对类型不分配空间，只对变量分配空间
- (2) 结构体类型中的成员名可以与程序中的变量名相同，但二者不代表同一对象
- (3) 对结构体变量中的成员（即“域”），可以单独使用，其作用与地位相当于普通变量

### 三、结构体变量的初始化和引用

例 9.1 把一个学生的信息放在一个结构体变量中并输出：

```
#include <stdio.h>
void main()
{
 struct student
 {
 int num;
 char name[20];
 char sex;
 char addr[20];
```



```

} student1 = {10101, "Li Lin", 'M', "123 Beijing Road"};
printf("NO.:%d\nname:%s\n, sex:%c\naddress:%s\n",
 student1.num, student1.name, student1.sex, student1.addr);
}

```

## 考点43. 结构体数组

### 一、定义结构体数组一般形式

- `struct 结构体名 { 成员表列 } 数组名[数组长度];`
- 或先声明一个结构体类型，再用此类型定义：`结构体类型 数组名[数组长度];`，如 `struct person leader[3];`

**二、对结构体数组初始化：**在定义数组的后面加 `= {初值表列};`，如 `struct person leader[3] = {"Li", 0, "Zhang", 0, "Sun", 0};`

**三、例 9.3** 有 3 个候选人，每个选民只能投票选一人，统计选票：

```

#include <string.h>
#include <stdio.h>
struct person
{
 char name[20];
 int count;
} leader[3] = {"Li", 0, "Zhang", 0, "Sun", 0};
void main()
{
 int i, j;
 char leader_name[20];
 for (i = 1; i <= 10; i++)
 {
 scanf("%s", leader_name);
 for (j = 0; j < 3; j++)
 if (strcmp(leader_name, leader[j].name) == 0)
 leader[j].count++;
 }
}

```

```

}

printf("\nResult : \n");
for (i = 0; i < 3; i++)
 printf("%5s:%d\n", leader[i].name, leader[i].count);
}

```

## 考点44. 用结构体变量和结构体变量的指针作函数参数

### 一、将一个结构体变量的值传递给另一个函数，有 3 个方法

- (1) 用结构体变量的成员作参数（如 `stu[1].num` 或 `stu[2].name`），属于"值传递"，实参与形参类型应一致
- (2) 用结构体变量作实参：将结构体变量所占内存单元内容全部按顺序传给形参（同类型结构体变量），空间和时间开销较大，改变形参不能返回主调函数，一般较少用
- (3) 用指向结构体变量（或数组）的指针作实参，将地址传给形参

### 二、例 9.7 有 N 个结构体变量 stu，内含学号、姓名和 3 门课程成绩，输出平均成绩最高的学生信息：

```

#include <stdio.h>

#define N 3

struct student
{
 int num;
 char name[20];
 float score[3];
 float aver;
};

void main()
{
 void input(struct student stu[]);
 struct student max(struct student stu[]);
 void print(struct student stu);
 struct student stu[N], *p = stu;
 input(p);
}

```

```

 print(max(p));
}

void input(struct student stu[])
{
 int i;
 printf("请输入各学生的信息：学号、姓名、三门课成绩:\n");
 for (i = 0; i < N; i++)
 {
 scanf("%d %s %f %f %f",
 &stu[i].num, stu[i].name,
 &stu[i].score[0], &stu[i].score[1], &stu[i].score[2]);
 stu[i].aver = (stu[i].score[0] + stu[i].score[1] + stu[i].score[2]) / 3.0;
 }
}

struct student max(struct student stu[])
{
 int i, m = 0;
 for (i = 0; i < N; i++)
 if (stu[i].aver > stu[m].aver) m = i;
 return stu[m];
}

void print(struct student stud)
{
 printf("\n 成绩最高的学生是:\n");
 printf("学号:%d\n 姓名:%s\n 三门课成绩:%5.1f,%5.1f,%5.1f\n 平均成绩:%6.2f\n",
 stud.num, stud.name, stud.score[0], stud.score[1], stud.score[2], stud.aver);
}

```

## 考点45. 共用体类型

- 概念：**有时想用同一段内存单元存放不同类型的变量。使几个不同的变量共享同一段内存的结构，称为"共用体"类型的结构
- 定义共用体类型变量的一般形式：**

union 共用体名

```
{
 成员表列
} 变量表列;
// 例如:
union data
{
 int i;
 char ch;
 float f;
} a, b, c;
```

1. **"共用体"与"结构体"的区别**：定义形式相似但含义不同。结构体变量所占内存长度是各成员占的内存长度之和，每个成员分别占有自己的内存单元；而共用体变量所占内存长度等于最长的成员的长度

## 考点46. 枚举类型

1. 如果一个变量只有几种可能的值，则可以定义为枚举类型。所谓"枚举"就是指把可能的值一一列举出来，变量的值只限于列举出来的值的范围内
2. 声明枚举类型用 `enum` 开头：

```
enum Weekday { sun, mon, tue, wed, thu, fri, sat };
enum Weekday workday, week_end;
workday = mon; // 正确
week_end = sun; // 正确
```

也可以直接定义枚举变量：`enum { sun, mon, tue, wed, thu, fri, sat } workday, week_end;`

其中 sun、mon、...、sat 称为枚举元素或枚举常量，它们是由用户定义的标识符。

## 第十章 利用文件保存数据

### 考点47. C文件的有关概念

## 一、什么是文件

文件有不同的类型，在程序设计中主要用到两种文件：

(1) **程序文件**：包括源程序文件（后缀 `.c`）、目标文件（`.obj`）、可执行文件（`.exe`）等，内容是程序代码

(2) **数据文件**：内容不是程序，而是供程序运行时读写的数据

- "文件"指存储在外部介质上数据的集合
- 操作系统是以文件为单位对数据进行管理
- C语言把文件看作是一个字符（或字节）的序列，一个输入输出流就是一个字符流或字节流
- C 的数据文件由一连串字符（或字节）组成，中间没有分隔符，对文件的存取以字符（字节）为单位，允许对文件存取一个字符，这种文件称为"流式文件"

## 二、文件名

文件标识包括 3 部分：（1）文件路径（2）文件名主干（3）文件后缀。如 `d:\cc\temp\file1.dat` 表示 `file1.dat` 存放在 d 盘 `cc` 目录下的 `temp` 子目录下。文件主干名遵循标识符命名规则，后缀一般不超过 3 个字母（`doc`、`txt`、`dat`、`c`、`cpp`、`obj`、`exe`、`ppt`、`bmp` 等）。

## 三、文件的分类

根据数据的组织形式，数据文件可分为 ASCII 文件和二进制文件：

- 数据在内存中以二进制形式存储，如果不加转换地输出到外存，就是二进制文件
- 如果要求在外存上以 ASCII 代码形式存储，则需要在存储前进行转换
- ASCII 文件又称文本文件，每一个字节放一个字符的 ASCII 代码

## 四、文件缓冲区

- ANSI C 标准采用"缓冲文件系统"处理数据文件
- 系统自动地在内存区为程序中每一个正在使用的文件开辟一个文件缓冲区
- 从内存向磁盘输出数据必须先送到缓冲区，装满缓冲区后才一起送到磁盘；从磁盘读入数据则一次充满缓冲区，再逐个送到程序数据区

## 五、文件类型指针

- 缓冲文件系统中关键的概念是"文件类型指针", 简称"文件指针"
- 每个被使用的文件都在内存中开辟一个相应的文件信息区, 存放文件的有关信息, 保存在一个结构体变量中, 类型由系统声明, 取名为 `FILE`
- `FILE` 结构体类型信息包含在头文件 `"stdio.h"` 中
- 一般设置一个指向 `FILE` 类型变量的指针变量: `FILE *fp1, *fp2, *fp3;`

## 考点48. 文件的打开与关闭

### 一、用 `fopen` 函数打开数据文件

- 调用方式: `fopen(文件名, 使用文件方式);`, 如 `fopen("a1", "r");` 表示打开名为 "a1" 的文件, 使用方式为"读入", 返回值是指向 a1 文件的指针
- 通常将返回值赋给一个指向文件的指针变量: `FILE *fp; fp = fopen("a1", "r");`
- 打开一个文件时通知编译系统 3 个信息: ①需要访问的文件的名字 ②使用文件的方式 ("读"还是"写"等) ③让哪一个指针变量指向被打开的文件
- 说明:

(1) 最基本的是 "r"、"w"、"a" 三种方式, 其后加 "b" 表示二进制文件, "+" 表示既可读又可写

(2) 如果不能实现"打开", `fopen` 将带回一个空指针值 `NULL`。常用下面的方法打开:

```
if ((fp = fopen("file1", "r")) == NULL)
{
 printf("cannot open this file\n");
 exit(0); // 终止正在执行的程序
}
```

### 二、用 `fclose` 函数关闭文件

`fclose(文件指针);`, 如 `fclose(fp);`。如果不关闭文件将会丢失数据。

考点49. 文件的顺序读写

一、向文件读写字符

| 函数名   | 调用形式          | 功能                  | 返回值                       |
|-------|---------------|---------------------|---------------------------|
| fgetc | fgetc(fp)     | 从 fp 指向的文件读入一个字符    | 读成功带回所读字符，失败返回 EOF(即 -1)  |
| fputc | fputc(ch, fp) | 把字符 ch 写到 fp 指向的文件中 | 写成功返回输出的字符，失败返回 EOF(即 -1) |

二、向文件读写一个字符串

| 函数名   | 调用形式              | 功能                                | 返回值                   |
|-------|-------------------|-----------------------------------|-----------------------|
| fgets | fgets(str, n, fp) | 从 fp 指向的文件读入长度为 (n-1) 的字符串存放到 str | 读成功返回地址 str，失败返回 NULL |
| fputs | fputs(str, fp)    | 将 str 所指向字符串写到 fp 指向的文件           | 写成功返回 0，否则返回非 0 值     |

说明：

(1) fgets(str, n, fp) 中 n 是要求得到的字符个数，但实际只读 n-1 个字符，然后在最后加一个 '\0'，共 n 个字符放到 str。如果在读完 n-1 个字符之前遇到换行符 \n 或 EOF，读入即结束，但将所遇到的 \n 也作为一个字符读入。执行成功返回 str 首地址，一开始就遇到文件尾或读错返回 NULL

(2) fputs 第一个参数可以是字符串常量、字符数组名或字符型指针，字符串末尾 '\0' 不输出。输出成功函数值为 0，失败为 EOF

三、文件的格式化读写

```
fprintf(文件指针, 格式字符串, 输出表列);
fscanf(文件指针, 格式字符串, 输入表列);
// 如:
```

```
fprintf(fp, "%d, %.2f", i, f);
fscanf(fp, "%d, %f", &i, &f);
```

## 四、用二进制方式读写文件

```
fread(buffer, size, count, fp);
fwrite(buffer, size, count, fp);
```

- `buffer`：是一个地址（对 `fread` 是存放读入数据的存储区地址；对 `fwrite` 是要输出数据的存储区起始地址）
- `size`：要读写的字节数
- `count`：要读写多少个数据项
- `fp`：FILE 类型指针

## 考点50. 文件的随机读写

### 一、文件位置标记及其定位

1. **文件位置标记**：为了对读写进行控制，系统为每个文件设置了一个位置指针，用来指示当前的读写位置

2. **文件位置指针的定位**：

(1) 用 `rewind` 函数使文件指针指向文件头：作用是使文件指针重新返回文件的开头，此函数没有返回值

(2) 用 `fseek` 函数移动位置指针，调用形式：`fseek(文件类型指针, 位移量, 起始点)`

- 起始点 0 代表"文件开始"(SEEK\_SET), 1 为"当前位置"(SEEK\_CUR), 2 为"文件末尾"(SEEK\_END)

- 位移量以起始点为基点向前移动的字节数，应为 long 型数据（数字末尾加 L）

- `fseek` 函数一般用于二进制文件：

```
fseek(fp, 100L, 0);
fseek(fp, 50L, 1);
fseek(fp, -10L, 2);
```



(3) 用 `ftell` 函数测定位置指针的当前位置：得到流式文件中位置指针的当前位置，用相对于文件开头的位移量表示。返回值为 `-1L` 表示出错：

```
i = ftell(fp);
if (i == -1L) printf("error\n");
```

---

## 第二部分 数据结构

### 第一章 绪论

#### 考点51. 数据结构的研究内容

##### 一、数据结构的研究内容

研究非数值计算的程序设计问题中计算机的操作对象以及它们之间的关系和操作。

##### 二、数据结构发展历程

1. 形成阶段：60 年代初期，"数据结构"有关的内容散见于操作系统、编译原理和表处理语言等课程。1968 年，"数据结构"被列入美国一些大学计算机科学系的教学计划
2. 发展阶段：数据结构的概念不断扩充，包括了网络、集合代数论、关系等"离散数学结构"的内容。70 年代后期，我国高校陆续开设该课程

#### 考点52. 基本概念和术语

##### 一、基本概念和术语

1. **数据(data)**：所有能输入到计算机中去的描述客观事物的符号（数值性数据、非数值性数据）

2. **数据元素(data element)**: 数据的基本单位, 也称结点(node)或记录(record)

3. **数据项(data item)**: 有独立含义的数据最小单位, 也称域(field)

- 三者关系: 数据 > 数据元素 > 数据项 (例: 学生表 > 个人记录 > 学号、姓名.....)

1. **数据对象(Data Object)**: 相同特性数据元素的集合, 是数据的一个子集 (如学生数据对象=学生记录集合; 整数数据对象  $N=\{0,1,2,\dots\}$ )

2. **数据结构(Data Structure)**: 相互之间存在一种或多种特定关系的数据元素的集合。"结构"就是指数据元素之间存在的关系

## 二、数据结构的两个层次

1. **逻辑结构**: 数据元素间抽象化的相互关系, 与数据的存储无关, 独立于计算机, 是从具体问题抽象出来的数学模型

- 划分方法一: 线性结构 (有且仅有一个开始和一个终端结点, 且所有结点都最多只有一个直接前趋和一个后继, 如线性表、栈、队列、串); 非线性结构 (一个结点可能有多个直接前趋和直接后继, 如树、图)

- 划分方法二: 集合 (除"同属于一个集合"外无其它关系); 线性结构 (一个对一个); 树形结构 (一个对多个); 图形/网状结构 (多个对多个)

1. **存储结构 (物理结构)**: 数据元素及其关系在计算机存储器中的存储方式

- 顺序存储结构: 借助元素在存储器中的相对位置来表示数据元素间的逻辑关系

- 链式存储结构: 借助指示元素存储地址的指针表示数据元素间的逻辑关系

**三、数据的运算**: 逻辑结构和存储结构都相同, 但运算不同, 则数据结构不同 (例如栈与队列)。对于一种数据结构, 常见的运算: 插入、删除、修改、查找、排序。

## 四、数据类型

1. 定义: 在一种程序设计语言中, 变量所具有的数据种类

2. C语言基本数据类型: `char`、`int`、`float`、`double`、`void`

3. 构造数据类型: 数组、结构体、共用体、文件

- 数据类型是一组性质相同的值的集合, 以及定义于这个集合上的一组运算的总称

**五、抽象数据类型**: 更高层次的数据抽象, 由用户定义, 用以表示应用问题的数据模型, 由基本的数据类型组成, 并包括一组相关的操作。

```
ADT 抽象数据类型名 {
 数据对象: <数据对象的定义>
 数据关系: <数据关系的定义>
 基本操作: <基本操作的定义>
} ADT 抽象数据类型名
```

## 考点53. 算法与算法分析

**一、算法定义：**一个有穷的指令集，这些指令为解决某一特定任务规定了一个运算序列。

**二、算法的描述：**1. 自然语言 2. 流程图 3. N-S 流程图 4. 程序设计语言 5. 伪码

**三、算法的特性：**1. 输入（有 0 个或多个输入）2. 输出（有一个或多个输出）3. 确定性（每步定义确切、无歧义）4. 有穷性（执行有穷步后结束）5. 有效性（每一条运算足够基本）

**四、算法的评价：**1. 正确性 2. 可读性 3. 健壮性 4. 高效性（时间代价和空间代价）

**五、算法效率的度量：**用依据该算法编制的程序在计算机上执行所消耗的时间来度量。两种方法：

1. 事后统计：利用计算机内的计时功能，不同算法的程序可以用一组或多组相同的统计数据区分
2. 事前分析估计：一个高级语言程序在计算机上运行所消耗的时间取决于：依据的算法选用何种策略、问题的规模、程序语言、编译程序产生机器代码质量、机器执行指令速度

## 六、问题规模与语句频度

1. **问题规模：**影响算法时间代价的主要原因，是算法求解问题输入量的多少，一般用  $n$  表示。 $n$  越大算法执行时间越长（排序  $n$  为记录数；矩阵  $n$  为阶数；多项式  $n$  为项数；集合  $n$  为元素个数；树  $n$  为结点个数；图  $n$  为顶点数或边数）
2. **语句频度：**一条语句的重复执行次数。一个算法的执行时间可用该算法中所有语句频度之和来度量
3. 例 1： $N \times N$  矩阵相乘

```

for(i=1; i<=n; i++) // 频度 n+1
 for(j=1; j<=n; j++) // 频度 n*(n+1)
 {
 c[i][j]=0; // 频度 n²
 for(k=1; k<=n; k++) // 频度 n²*(n+1)
 c[i][j]=c[i][j]+a[i][k]*b[k][j]; // 频度 n³
 }

```

执行时间与执行频度  $f(n)$  成正比:  $f(n) = 2n^3 + 3n^2 + 2n + 1$

## 七、时间复杂度定义

1. 算法中基本语句重复执行的次数是问题规模  $n$  的某个函数  $f(n)$ ，算法的时间量度记作  $T(n)=O(f(n))$ ，称渐近时间复杂度
2. 时间复杂度分析：找出语句频度最大的那条语句作为基本语句，计算其频度得到  $f(n)$ ，取其数量级用符号 "O" 表示。注意：时间复杂度是由嵌套最深层语句的频度决定的

## 八、时间复杂度举例

(1) `{x++; s=0;}` 两条语句频度均为 1， $T(n)=O(1)$ ，常量阶。改为 `for(i=0; i<10000; i++) {x++; s=0;}` 时间复杂度仍为  $O(1)$

(2) `for(i=0; i<n; i++) {x++; s=0;}` 频度  $f(n)=n$ ， $T(n)=O(n)$ ，线性阶

(3)

```

x = 0; y = 0;
for (int k = 0; k < n; k++) x++;
for (int i = 0; i < n; i++)
 for (int j = 0; j < n; j++) y++; // 频度最大 f(n)=n²

```

$T(n)=O(n^2)$ ，平方阶。一般考虑最内层循环。

(4)  $n \times n$  阶矩阵加法：

```
for(i=0; i<n; i++)
 for(j=0; j<n; j++)
 c[i][j] = a[i][j] + b[i][j];
```

频度  $n*n$ ,  $T(n)=O(n^2)$

(5)

```
for(i=1; i<=n; i++)
 for(j=1; j<=i; j++)
 for(k=1; k<=j; k++)
 x = x + 1;
```

$T(n)=O(n^3)$ ，立方阶

(6) `for(i=1; i<n; i=i*2){x++;s=0;}` 频度  $f(n)$  满足  $2^{f(n)} \leq n$ , 即  $f(n) \leq \log_2 n$ ,  $T(n)=O(\log_2 n)$ ，对数阶

**总结：**时间复杂度  $T(n)$  按数量级递增顺序为  $O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < \dots$  (指数阶)

## 九、最好、最坏和平均时间复杂度

例 4 顺序查找，在数组  $a[i]$  中查找值等于  $e$  的元素：

```
for (i=0; i<n; i++)
```

```
 if (a[i]==e) return i+1;
return 0;
```

最好情况 1 次，最坏情况  $n$ ，平均  $n/2$ ，平均时间复杂度  $O(n)$

## 十、算法空间复杂度

空间复杂度：算法所需存储空间的度量，记作  $S(n)=O(f(n))$ ，其中  $n$  为问题规模。算法要占据的空间包括：算法本身、输入/输出、指令、常数、变量等，以及算法要使用的辅助空间。

例 5 将一维数组  $a$  中的  $n$  个数逆序存放到原数组中：

- 【算法1】原地工作  $S(n)=O(1)$

```
for(i=0; i<n/2; i++)
{
 t = a[i];
 a[i] = a[n-i-1];
 a[n-i-1] = t;
}
```

- 【算法2】借助辅助数组  $b$   $S(n)=O(n)$

```
for(i=0; i<n; i++) b[i] = a[n-i-1];
for(i=0; i<n; i++) a[i] = b[i];
```

## 第二章 线性表

### 考点54. 线性表的定义和特点

**一、线性表的定义：**由  $n$  个特性相同的数据元素构成的有限序列。

## 二、线性表的特点

- (1) 存在唯一的一个被称作"第一个"的数据元素
- (2) 存在唯一的一个被称作"最后一个"的数据元素
- (3) 除第一个之外，结构中每个元素都有一个前驱
- (4) 除最后一个之外，结构中每个元素都有一个后继

## 考点55. 线性表的顺序表示和实现

### 一、顺序表的定义

线性表的顺序表示又称为顺序存储结构或顺序映像。把逻辑上相邻的数据元素存储在物理上相邻的存储单元中的存储结构，简言之，逻辑上相邻，物理上也相邻。用一组地址连续的存储单元依次存储线性表的元素，可通过数组 `V[n]` 实现。

**二、顺序表的操作实现：**1. 初始化线性表 2. 取值（根据位置  $i$  获取相应位置数据元素的内容）3. 查找（根据指定数据获取数据所在的位置）4. 插入（插在第  $i$  个结点之前）5. 删除（删除第  $i$  个结点）

### 三、顺序表（顺序存储结构）的特点

- 1. 利用数据元素的存储位置表示线性表中相邻数据元素之间的前后关系，即线性表的逻辑结构与存储结构一致
- 2. 在访问线性表时，可以快速地计算出任何一个数据元素的存储地址，访问每个元素所花时间相等——这种存取方法被称为**随机存取法**

### 四、顺序表的优缺点

- 优点：存储密度大；可以随机存取表中任一元素
- 缺点：在插入、删除某一元素时需要移动大量元素；浪费存储空间；属于静态存储形式，数据元素的个数不能自由扩充

## 考点56. 线性表的链式表示和实现

### 一、链表的定义

链式存储结构中，结点在存储器中的位置是任意的，即逻辑上相邻的数据元素在物理上不一定相邻（非顺序映像或链式映像）。通过指针来实现，各结点由两个域组成：数据域（存储元素数值数据）和指针域（存储直接后继结点的存储位置）。

### 二、链式存储有关的术语

1. 单链表、双链表、循环链表：

- 1) 结点只有一个指针域的链表，称为单链表或线性链表
- 2) 有两个指针域的链表，称为双链表
- 3) 首尾相接的链表称为循环链表

1. 头指针、头结点、首元结点：

- 1) 头指针是指向链表中第一个结点的指针
- 2) 头结点是在链表的首元结点之前附设的一个结点；数据域内只放空表标志和表长等信息
- 3) 首元结点是指链表中存储第一个数据元素  $a_1$  的结点

**三、重要操作：**1. 初始化(构造一个空表) 2. 取值 3. 查找 4. 插入（将值为  $x$  的新结点插入到表的第  $i$  个结点的位置上，即插入到  $a_{i-1}$  与  $a_i$  之间） 5. 删除（删除第  $i$  个结点）

### 四、链式存储结构的特点

1. 结点在存储器中的位置是任意的，即逻辑上相邻的数据元素在物理上不一定相邻
2. 访问时只能通过头指针进入链表，并通过每个结点的指针域向后扫描其余结点，寻找第一个结点和最后一个结点所花费的时间不等——这种存取方法被称为**顺序存取法**

### 五、链表的优缺点

- 优点：数据元素的个数可以自由扩充；插入、删除等操作不必移动数据，只需修改链接指针，修改效率较高



- 缺点：存储密度小；存取效率不高，必须采用顺序存取（顺藤摸瓜）

考点57. 循环链表

对循环链表，有时不给出头指针，而给出尾指针；可以更方便地找到第一个和最后一个结点。

考点58. 双向链表

1. 双向链表的插入（源稿此处仅有标题，未展开具体步骤，详见文末说明）

考点59. 顺序表和链表的比较

| 比较项目  | 顺序表                               | 链表                   |
|-------|-----------------------------------|----------------------|
| 存储空间  | 预先分配，会导致空间闲置或溢出                   | 动态分配，不会出现空间闲置或溢出     |
| 存储密度  | 不用为表示逻辑关系增加额外开销，存储密度等于 1          | 需借助指针体现逻辑关系，存储密度小于 1 |
| 存取元素  | 随机存取，按位置访问元素 $O(1)$               | 顺序存取，按位置访问元素 $O(n)$  |
| 插入、删除 | 平均移动约表中一半元素， $O(n)$               | 不需移动元素，确定位置后 $O(1)$  |
| 适用情况  | ①表长变化不大且能事先确定范围 ②很少插入删除，经常按位置序号访问 | ①长度变化较大 ②频繁进行插入或删除操作 |

第三章 栈和队列

考点60. 栈和队列的定义和特点

一、栈

1. 定义：只能在表的一端（栈顶）进行插入和删除运算的线性表
2. 逻辑结构：与线性表相同，仍为一对一关系

- 3. 存储结构：用顺序栈或链栈存储均可，但以顺序栈更常见
- 4. 运算规则：只能在栈顶运算，且访问结点时依照**后进先出（LIFO）**或先进后出（FILO）的原则
- 5. 实现方式：关键是编写入栈和出栈函数，基本操作有入栈、出栈、读栈顶元素值、建栈、判断栈满、栈空等

二、队列

- 1. 定义：队列是一种**先进先出（FIFO）**的线性表，在表一端插入，在另一端删除
- 2. 逻辑结构：与线性表相同，仍为一对一关系
- 3. 存储结构：用顺序队列或链队存储均可
- 4. 运算规则：先进先出（FIFO）
- 5. 实现方式：关键是编写入队和出队函数

三、栈、队列与一般线性表的区别

栈、队列是一种特殊（操作受限）的线性表，区别仅在于运算规则不同。

|       | 逻辑结构 | 存储结构   | 运算规则    |
|-------|------|--------|---------|
| 一般线性表 | 一对一  | 顺序表、链表 | 随机、顺序存取 |
| 栈     | 一对一  | 顺序栈、链栈 | 后进先出    |
| 队列    | 一对一  | 顺序队、链队 | 先进先出    |

第四章 串、数组、广义表

考点61. 串的定义

- 1. **串(String)**：零个或多个字符组成的有限序列
- 2. **子串**：串中任意连续字符组成的子序列
- 3. **主串**：包含子串的串
- 4. **字符位置**：字符在序列中的序号
- 5. **子串位置**：子串的的第一个字符在主串出现的位置
- 6. **串相等**：两个串的值相等
- 7. **空格串**：零个字符的串

如 `a="BEI"`、`b="JING"`、`c="BEIJING"`、`d="BEI JING"`

## 考点62. 广义表

### 一、广义表的定义

广义表（列表）： $n(n \geq 0)$  个表元素组成的有限序列，记作  $LS = (a_0, a_1, a_2, \dots, a_{n-1})$ 。LS 是表名， $a_i$  是表元素，它可以是表（称为子表），可以是数据元素（称为原子）。n 为表的长度。n=0 的广义表为空表。

### 二、广义表与线性表的区别

1. 线性表的成分都是结构上不可分的单元素
2. 广义表的成分可以是单元素，也可以是有结构的表
3. 线性表是一种特殊的广义表
4. 广义表不一定是线性表，也不一定是线性结构

### 三、广义表的基本运算

1. **求表头 GetHead(L)**：非空广义表的第一个元素，可以是一个单元素，也可以是一个子表
2. **求表尾 GetTail(L)**：非空广义表除去表头元素以外其它元素所构成的表。表尾一定是一个表

- `A=()`：GetHead 和 GetTail 均无定义
- `A=(a, b)`：GetHead(A)=a, GetTail(A)=(b)
- `A=(a)`：GetHead(A)=a, GetTail(A)=()
- `A=((a))`：GetHead(A)=(a), GetTail(A)=()

### 四、广义表的特点

1. 有次序性：一个直接前驱和一个直接后继
2. 有长度：= 表中元素个数
3. 有深度：= 表中括号的重数
4. 可递归：自己可以作为自己的子表
5. 可共享：可以为其他广义表所共享

## 第五章 树和二叉树

### 考点63. 树和二叉树的定义

#### 一、树的定义

树 (Tree) 是  $n$  ( $n \geq 0$ ) 个结点的有限集, 它或为空树 ( $n=0$ ) ; 或为非空树, 对于非空树  $T$ :

1. 有且仅有一个称之为根的结点
2. 除根结点以外的其余结点可分为  $m$  ( $m > 0$ ) 个互不相交的有限集  $T_1, T_2, \dots, T_m$ , 其中每一个集合本身又是一棵树, 并且称为根的子树 (SubTree)

#### 二、树的基本术语

1. 根: 根结点 (没有前驱)
2. 叶子: 终端结点 (没有后继)
3. 森林: 指  $m$  棵不相交的树的集合 (例如删除  $A$  后的子树个数)
4. 有序树: 结点各子树从左至右有序, 不能互换 (左为第一)
5. 无序树: 结点各子树可互换位置
6. 双亲: 上层的那个结点 (直接前驱)
7. 孩子: 下层结点的子树的根 (直接后继)
8. 兄弟: 同一双亲下的同层结点
9. 堂兄弟: 双亲位于同一层的结点 (但并非同一双亲)
10. 祖先: 从根到该结点所经分支的所有结点
11. 子孙: 该结点下层子树中的任一结点
12. 结点: 树的数据元素
13. 结点的度: 结点挂接的子树数
14. 结点的层次: 从根到该结点的层数 (根结点算第一层)
15. 终端结点: 度为 0 的结点, 即叶子
16. 分支结点: 度不为 0 的结点 (也称为内部结点)
17. 树的度: 所有结点度中的最大值
18. 树的深度 (或高度): 所有结点中最大的层数

#### 三、二叉树的定义

二叉树 (Binary Tree) 是  $n$  ( $n \geq 0$ ) 个结点所构成的集合, 它或为空树 ( $n=0$ ) ; 或为非空树, 对于非空树  $T$ :

- (1) 有且仅有一个称之为根的结点
- (2) 除根结点以外的其余结点分为两个互不相交的子集  $T_1$  和  $T_2$ , 分别称为  $T$  的左子树和右子树, 且  $T_1$  和  $T_2$  本身又都是二叉树

为何要重点研究每结点最多只有两个"叉"的树? 二叉树的结构最简单, 规律性最强; 可以证明, 所有树都能转为唯一对应的二叉树, 不失一般性。

**四、二叉树基本特点:** 结点的度小于等于 2; 有序树 (子树有序, 不能颠倒)。

## 考点64. 二叉树的性质和存储结构

### 一、二叉树的性质

- **性质1:** 在二叉树的第  $i$  层上至多有  $2^{(i-1)}$  个结点 ( $i \geq 1$ ), 至少 1 个结点
  - **性质2:** 深度为  $k$  的二叉树至多有  $2^k - 1$  个结点 ( $k \geq 1$ ), 至少  $k$  个结点
  - **性质3:** 对于任何一棵二叉树, 若 2 度的结点数有  $n_2$  个, 则叶子数  $n_0$  必定为  $n_2 + 1$  (即  $n_0 = n_2 + 1$ )
- **满二叉树:** 一棵深度为  $k$  且有  $2^k - 1$  个结点的二叉树 (特点: 每层都"充满"了结点)
- **完全二叉树:** 深度为  $k$  的、有  $n$  个结点的二叉树, 当且仅当其每一个结点都与深度为  $k$  的满二叉树中编号从 1 至  $n$  的结点——对应。满二叉树是叶子一个也不少的树; 完全二叉树虽然前  $n-1$  层是满的, 但最底层允许在右边缺少连续若干个结点。满二叉树是完全二叉树的一个特例
- **性质4:** 具有  $n$  个结点的完全二叉树的深度必为  $\lfloor \log_2 n \rfloor + 1$
  - **性质5:** 一棵有  $n$  个结点的完全二叉树, 若从上至下、从左至右编号, 对于任意结点  $1 \leq i \leq n$ :
- (1)  $i=1$  时, 结点 1 是二叉树的根, 无双亲;  $i>1$  时, 其双亲的编号必为  $\lfloor i/2 \rfloor$
  - (2) 若  $2i > n$ , 结点  $i$  无左孩子; 否则其左孩子编号必为  $2i$
  - (3) 若  $2i+1 > n$ , 结点  $i$  无右孩子; 否则其右孩子编号必为  $2i+1$

### 二、二叉树的存储

1. **顺序存储:** 按完全二叉树的结点层次编号, 依次存放二叉树中的数据元素。一般二叉树使用顺序存储会浪费存储空间

2. **链式存储**：1) 二叉链表 2) 三叉链表

## 考点65. 遍历二叉树和线索二叉树

### 一、遍历二叉树

1. **遍历定义**：指按某条搜索路线遍访每个结点且不重复（又称周游）

2. **遍历用途**：它是树结构插入、删除、修改、查找和排序运算的前提，是二叉树一切运算的基础和核心

3. **遍历规则**：

- DLR—先序遍历，即先根再左再右

- LDR—中序遍历，即先左再根再右

- LRD—后序遍历，即先左再右再根

1. 用二叉树表示算术表达式（示例遍历结果）：

- 先序遍历（前缀表示）： $+ * * / A B C D E$

- 中序遍历（中缀表示）： $A / B * C * D + E$

- 后序遍历（后缀表示）： $A B / C * D * E +$

- 层序遍历： $+ * E * D / C A B$

说明：源稿此处配有树形结构示意图，文字版无法还原具体树形，仅保留各遍历结果。

## 考点66. 哈夫曼树及其应用

### 一、基本概念

1. **路径**：由一结点到另一结点间的分支所构成

2. **路径长度**：路径上的分支数目

3. **带权路径长度**：结点到根的路径长度与结点上权的乘积

4. **树的带权路径长度**：树中所有叶子结点的带权路径长度之和

5. **哈夫曼树**：带权路径长度最小的树，也称最优二叉树

## 二、哈夫曼树的构造

- 根据给定的  $n$  个权值  $\{w_1, w_2, \dots, w_n\}$ ，构造  $n$  棵只有根结点的二叉树
- 在森林中选取两棵根结点权值最小的树作左右子树，构造一棵新的二叉树，置新二叉树根结点权值为其左右子树根结点权值之和
- 在森林中删除这两棵树，同时将新得到的二叉树加入森林中
- 重复上述两步，直到只含一棵树为止，这棵树即哈夫曼树

一棵有  $n$  个叶子结点的 Huffman 树有  $2n-1$  个结点。（例如 7、5、2、4 四个结点构造哈夫曼树，源稿配有示意图）

## 三、哈夫曼树应用实例——哈夫曼编码

采用二叉树设计前缀编码，出现概率最高的离根最近。左分支用"0"，右分支用"1"。

例：ABACCDA  $\rightarrow$  A—0, B—110, C—10, D—111，编码为 0110010101110

### 哈夫曼编码的几点结论：

1. 哈夫曼编码是不等长编码
2. 哈夫曼编码是前缀编码，即任一字符的编码都不是另一字符编码的前缀
3. 哈夫曼编码树中没有度为 1 的结点。若叶子结点的个数为  $n$ ，则哈夫曼编码树的结点总数为  $2n-1$
4. 发送过程：根据由哈夫曼树得到的编码表送出字符数据
5. 接收过程：按左 0、右 1 的规定，从根结点走到一个叶结点，完成一个字符的译码。反复此过程，直到接收数据结束

## 第六章 图

### 考点67. 图的定义和基本术语

#### 一、图的定义和术语

a) **图**：Graph= $(V, E)$ ， $V$  是顶点（数据元素）的有穷非空集合； $E$  是边的有穷集合

b) **无向图**：每条边都是无方向的

- c) **有向图**：每条边都是有方向的
- d) **子图**：设有两个图  $G=(V,\{E\})$ 、 $G_1=(V_1,\{E_1\})$ ，若  $V_1\subseteq V$ ， $E_1\subseteq E$ ，则称  $G_1$  是  $G$  的子图
- e) **完全图**：任意两个点都有一条边相连。无向完全图  $n(n-1)/2$  条边；有向完全图  $n(n-1)$  条边
- f) **稀疏图**：有很少边或弧的图
- g) **密图**：有较多边或弧的图
- h) **网**：边/弧带权的图
- i) **邻接**：有边/弧相连的两个顶点之间的关系。存在  $(v_i,v_j)$  则称  $v_i$  和  $v_j$  互为邻接点；存在  $\langle v_i,v_j \rangle$  则称  $v_i$  邻接到  $v_j$ ， $v_j$  邻接于  $v_i$
- j) **关联(依附)**：边/弧与顶点之间的关系
- k) **顶点的度**：与该顶点相关联的边的数目，记  $TD(v)$
- l) 在有向图中，顶点的度等于该顶点的入度与出度之和：
- 顶点  $v$  的入度是以  $v$  为终点的有向边的条数，记  $ID(v)$
  - 顶点  $v$  的出度是以  $v$  为始点的有向边的条数，记  $OD(v)$
- m) **路径**：接续的边构成的顶点序列
- n) **路径长度**：路径上边或弧的数目/权值之和
- o) **回路(环)**：第一个顶点和最后一个顶点相同的路径
- p) **简单路径**：除路径起点和终点可以相同外，其余顶点均不相同的路径
- q) **简单回路(简单环)**：除路径起点和终点相同外，其余顶点均不相同的路径
- r) **连通图 (强连通图)**：若对任何两个顶点  $v$ 、 $u$  都存在从  $v$  到  $u$  的路径，则称  $G$  是连通图 (强连通图)
- s) **连通分量 (强连通分量)**：无向图  $G$  的极大连通子图
- t) **极小连通子图 (连通图的生成树)**：该子图是  $G$  的连通子图，在该子图中删除任何一条边，子图不再连通
- u) **生成树**：包含无向图  $G$  所有顶点的极小连通子图。**生成森林**：对非连通图，由各个连通分量的生成树的集合

## 考点68. 图的存储结构

### 一、顺序存储结构：数组表示法 (邻接矩阵)



- 建立一个顶点表（记录各个顶点信息）和一个邻接矩阵（表示各个顶点之间关系）
  - 设图  $A=(V,E)$  有  $n$  个顶点，则图的邻接矩阵是一个二维数组  $A.Edge[n][n]$
1. **无向图的邻接矩阵**：矩阵是对称的；顶点  $i$  的度 = 第  $i$  行(列)中 1 的个数；完全图的对角元素为 0，其余 1
  2. **有向图的邻接矩阵**：第  $i$  行含义是  $v_i$  的出度；第  $i$  列含义是  $v_i$  的入度。矩阵可能是不对称的；顶点的出度=第  $i$  行元素之和；入度=第  $i$  列元素之和；度=第  $i$  行元素之和+第  $i$  列元素之和
  3. **网（即有权图）的邻接矩阵**： $A.Edge[i][j] = W_{ij}$ （若  $\langle v_i, v_j \rangle$  或  $(v_i, v_j) \in VR$ ）， $= \infty$ （无边/弧）
  4. **邻接矩阵表示法的特点**：优点是容易实现图的操作（求某顶点的度、判断顶点间是否有边、找邻接点等）；缺点是  $n$  个顶点需要  $n*n$  个单元存储边，空间效率为  $O(n^2)$ ，对稀疏图尤其浪费空间

## 二、邻接表（链式）表示法

对每个顶点  $v_i$  建立一个单链表，把与  $v_i$  有关联的边的信息链接起来，每个结点设为 3 个域；每个单链表的头结点另外用顺序存储结构存储（设 2 个域，存  $v_i$  信息）。

1. **有向图的邻接表**：出度  $OD(v_i) =$  单链出边表中链接的结点数；入度  $ID(v_i) =$  邻接点域为  $v_i$  的弧个数；度  $TD(v_i) = OD(v_i) + ID(v_i)$
2. **无向图的邻接表**：邻接表不唯一（因各个边结点的链入顺序是任意的）； $TD(v_i) =$  单链表中链接的结点个数
3. **邻接表表示法的特点**：优点是空间效率高，容易寻找顶点的邻接点，便于删除和增加结点，便于统计边的数目；缺点是判断两顶点间是否有边或弧需搜索两结点对应的单链表，没有邻接矩阵方便，不便于计算有向图各个顶点的度

## 考点69. 图的遍历

### 一、深度优先搜索（DFS — Depth\_First Search）

基本思想：类似树的先序遍历过程，构成一棵以  $v_1$  为根的树，称为深度优先生成树。

- 用邻接矩阵来表示图，遍历图中每一个顶点都要从头扫描该顶点所在行，时间复杂度为  $O(n^2)$
- 用邻接表来表示图，虽然有  $2e$  个表结点，但只需扫描  $e$  个结点即可完成遍历，加上访问  $n$  个头结点的时间，时间复杂度为  $O(n+e)$
- **结论**：稠密图适于在邻接矩阵上进行深度遍历；稀疏图适于在邻接表上进行深度遍历

### 二、广度优先搜索（BFS — Breadth\_First Search）

基本思想：类似树的层次遍历过程，构成一棵以  $v_1$  为根的树，称为广度优先生成树。简单归纳：在访问了起始点  $v$  之后，依次访问  $v$  的邻接点；然后再依次访问这些顶点中未被访问过的邻接点；直到所有顶点都被访问过为止。

- 广度优先搜索是一种分层的搜索过程，每向前走一步可能访问一批顶点，不像深度优先搜索那样有回退的情况
- 因此，广度优先搜索不是一个递归的过程，其算法也不是递归的
- 使用邻接矩阵，BFS 对于每一个被访问到的顶点都要循环检测矩阵中的整整一行（ $n$  个元素），总的时间代价为  $O(n^2)$
- 用邻接表来表示图，只需扫描  $e$  个结点加  $n$  个头结点，时间复杂度为  $O(n+e)$

## 考点70. 图的应用

### 一、最小生成树

使用不同的遍历图的方法，可以得到不同的生成树；从不同的顶点出发，也可能得到不同的生成树。按照生成树的定义， $n$  个顶点的连通网络的生成树有  $n$  个顶点、 $n-1$  条边。目标：在网的多个生成树中，寻找一个各边权值之和最小的生成树，即最小生成树。

如何求最小生成树：

- **Prim（普里姆）算法**：归并顶点，与边数无关，适于稠密网
- **Kruskal（克鲁斯卡尔）算法**：归并边，适于稀疏网

## 第七章 查找

### 考点71. 查找的基本概念

1. **查找表**：由同一类型的数据元素（或记录）构成的集合
2. **静态查找表**：查找的同时对查找表不做修改操作
3. **动态查找表**：查找的同时对查找表具有修改操作（如插入和删除）
4. **关键字**：记录中某个数据项的值，可用于识别一个记录
5. **主关键字**：唯一标识数据元素
6. **次关键字**：可以标识若干个数据元素
7. **关键字的平均比较次数**，也称平均搜索长度 ASL（Average Search Length）： $ASL = \sum_{i=1..n} p_i \cdot c_i$

- n: 记录的个数
- $p_i$ : 查找第  $i$  个记录的概率 (通常认为  $p_i = 1/n$ )
- $c_i$ : 找到第  $i$  个记录所需的比较次数

## 考点72. 折半查找

1. 折半查找又称为二分查找，效率高，适用于顺序存储结构。要求线性表必须采用顺序存储结构，并且表中的元素按关键字有序排列
2. **算法步骤**: 设表长为  $n$ ,  $low$ 、 $high$  和  $mid$  分别指向待查元素所在区间的上界、下界和中点,  $k$  为给定值
  - 初始时, 令  $low=1, high=n$
  - $mid = (low+high)/2$
  - 若  $k==R[mid].key$ , 查找成功
  - 若  $k<R[mid].key$ , 则  $high=mid-1$
  - 若  $k>R[mid].key$ , 则  $low=mid+1$
  - 重复上述操作, 直至  $low>high$  时, 查找失败

例: 已知 11 个数据元素的有序表如下, 查找 27: (5, 16, 20, 27, 30, 36, 44, 55, 60, 67, 71)

1. **性能分析—判定树**:  $ASL = 1/11 \times (1 \times 1 + 2 \times 2 + 4 \times 3 + 4 \times 4) = 33/11 = 3$
2. **性能**: 查找过程每次将待查记录所在区间缩小一半, 比顺序查找效率高, 时间复杂度  $O(\log_2 n)$ 。适用条件: 采用顺序存储结构的有序表, 不宜用于链式结构

## 考点73. 哈希表的查找

### 一、哈希表的查找

- 基本思想: 记录的存储位置与关键字之间存在对应关系,  $Loc(i) = H(key_i)$
- 优点: 查找速度极快  $O(1)$ , 查找效率与元素个数  $n$  无关
- 例: 将 2001011810201 的所有信息存入  $V[01]$  单元; 将 2001011810202 存入  $V[02]$  单元.....将 2001011810231 存入  $V[31]$  单元。查找 2001011810216 的信息, 可直接访问  $V[16]$

## 二、相关术语

1. **哈希方法(杂凑法)**: 选取某个函数, 依该函数按关键字计算元素的存储位置并存放; 查找时, 由同一个函数对给定值  $k$  计算地址, 将  $k$  与地址单元中元素关键码进行比较, 确定查找是否成功
2. **冲突**: 不同的关键码映射到同一个哈希地址 ( $key1 \neq key2$ , 但  $H(key1) = H(key2)$ )
3. **同义词**: 具有相同函数值的两个关键字

## 三、哈希函数的构造方法 (根据元素集合的特性构造, 地址空间尽量小、均匀)

1. 数字分析法
2. 平方取中法
3. 折叠法
4. 除留余数法

## 四、处理冲突的方法

1. **开放定址法 (开地址法)**: 基本思想是有冲突时就去寻找下一个空的哈希地址, 只要哈希表足够大, 空的哈希地址总能找到, 并将数据元素存入。几种方法:

(1) **线性探测法**:  $H_i = (\text{Hash}(\text{key}) + d_i) \bmod m$  ( $1 \leq i < m$ ), 其中  $m$  为哈希表长度,  $d_i$  为增量序列  $1, 2, \dots, m-1$ , 且  $d_i = i$ ; 一旦冲突, 就找下一个空地址存入

例: 关键码集为  $\{47, 7, 29, 11, 16, 92, 22, 8, 3\}$ , 设哈希表表长  $m=11$ , 哈希函数  $\text{Hash}(\text{key}) = \text{key} \bmod 11$

(2) **二次探测法**:  $H_i = (\text{Hash}(\text{key}) \pm d_i) \bmod m$ , 其中  $d_i$  为增量序列  $1^2, -1^2, 2^2, -2^2, \dots, q^2$

(3) **伪随机探测法** (略)

1. **链地址法 (拉链法)**: 相同哈希地址的记录链成一单链表,  $m$  个哈希地址就设  $m$  个单链表, 然后用一个数组将  $m$  个单链表的表头指针存储起来, 形成一个动态的结构

例:  $(19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79)$ ,  $H(\text{key}) = \text{key} \% 13$

## 第八章 排序

### 考点74. 基本概念

1. **什么是排序**: 将一组杂乱无章的数据按一定规律顺次排列起来
2. **排序的目的**: 便于查找
3. **排序算法的好坏如何衡量**:

- 时间效率：排序速度（比较次数与移动次数）
  - 空间效率：占内存辅助空间的大小
  - 稳定性：A 和 B 的关键字相等，排序后 A、B 的先后次序保持不变，则称这种排序算法是稳定的
1. **内部排序 vs 外部排序**：若待排序记录都在内存中，称为内部排序；若待排序记录一部分在内存、一部分在外存，则称为外部排序。外部排序时，要将数据分批调入内存来排序，中间结果还要及时放入外存，显然更复杂
2. **内部排序算法分类**：
- 规则不同：插入排序、交换排序、选择排序、归并排序、分配排序
  - 时间复杂度不同：简单排序  $O(n^2)$ 、先进排序  $O(n \log_2 n)$

本章讨论的排序算法均按顺序存储结构，且关键字均为整型。

考点75. 插入排序

1. **基本思想**：每步将一个待排序的对象，按其关键码大小，插入到前面已经排好序的一组对象的适当位置上，直到对象全部插入为止。即边插入边排序，保证子序列中随时都是排好序的
2. **直接插入排序（基于顺序查找）**：整个排序过程为  $n-1$  趟插入，即先将序列中第 1 个记录看成一个有序子序列，然后从第 2 个记录开始，逐个进行插入，直至整个序列有序

例 (13, 6, 3, 31, 9, 27, 5, 11)：

| 步骤   | 序列                           |
|------|------------------------------|
| 原始序列 | [13], 6, 3, 31, 9, 27, 5, 11 |
| 第一趟  | [6, 13], 3, 31, 9, 27, 5, 11 |
| 第二趟  | [3, 6, 13], 31, 9, 27, 5, 11 |
| 第三趟  | [3, 6, 13, 31], 9, 27, 5, 11 |
| 第四趟  | [3, 6, 9, 13, 31], 27, 5, 11 |
| 第五趟  | [3, 6, 9, 13, 27, 31], 5, 11 |
| 第六趟  | [3, 5, 6, 9, 13, 27, 31], 11 |
| 第七趟  | [3, 5, 6, 9, 11, 13, 27, 31] |

考点76. 起泡排序  $O(n^2)$

**基本思想：**每趟不断将记录两两比较，并按"前小后大"规则交换。比如 6 个元素需要比较 5 趟。

- 第一趟，从第一个元素开始两两比较，需要比较五次，将最大的找出来放在最后一个元素位置
- 第二趟，排序过程最后一个元素不参与，其余元素从第一个开始两两比较，需要比较四次，将次大的找出来放在倒数第二个元素的位置
- 依次类推（注：25\\* 表示第二个出现的 25）

例 (21, 25, 49, 25\\*, 16, 08)：

| 步骤   | 序列                       |
|------|--------------------------|
| 原始序列 | 21, 25, 49, 25\*, 16, 08 |
| 第一趟  | 21, 25, 25\*, 16, 08, 49 |
| 第二趟  | 21, 25, 16, 08, 25\*, 49 |
| 第三趟  | 21, 16, 08, 25, 25\*, 49 |
| 第四趟  | 16, 08, 21, 25, 25\*, 49 |
| 第五趟  | 08, 16, 21, 25, 25\*, 49 |

**总结：**时间复杂度  $O(n^2)$ 、空间复杂度  $O(1)$ 、是一种稳定的排序方法。

**优点：**每趟结束时，不仅能挤出一个最大值到最后面位置，还能同时部分理顺其他元素；一旦下趟没有交换，还可提前结束排序。

考点77. 快速排序  $O(n \log_2 n)$

**基本思想：**

- 任取一个元素（如第一个）为中心
- 所有比它小的元素一律前放，比它大的元素一律后放，形成左右两个子表

- 对各子表重新选择中心元素并依此规则调整，直到每个子表的元素只剩一个

例 (21, 25, 49, 25\\*, 16, 08) :

- 初始: 21, 25, 49, 25\\*, 16, 08
- 08, 16, 21, 25\\*, 49, 25 → 08, 16, 21, 25, 25\\*, 49

总结:

- 时间效率:  $O(n \log_2 n)$  — 每趟确定的元素呈指数增加
- 空间效率:  $O(\log_2 n)$  — 递归要用到栈空间
- 稳定性: 不稳定，可选任一元素为支点

考点78. 选择排序

1. **基本思想**: 每一趟在后面  $n-i+1$  个中选出关键码最小的对象，作为有序序列的第  $i$  个记录

例 (21, 25, 49, 25\\*, 16, 08) :

| 步骤  | 序列                       |
|-----|--------------------------|
| 初始  | 21, 25, 49, 25\*, 16, 08 |
| 第一趟 | 08, 25, 49, 25\*, 16, 21 |
| 第二趟 | 08, 16, 49, 25\*, 25, 21 |
| 第三趟 | 08, 16, 21, 25\*, 25, 49 |
| 第四趟 | 08, 16, 21, 25\*, 25, 49 |
| 第五趟 | 08, 16, 21, 25\*, 25, 49 |

总结: 时间复杂度  $O(n^2)$ 、空间复杂度  $O(1)$ 、稳定。

考点79. 排序算法比较

源稿此考点为对比图表，文字版未能还原具体对比内容，详见文末说明。综合前述各考点，常见内部排序算法复杂度与稳定性归纳如下：

| 排序算法   | 平均时间复杂度         | 空间复杂度         | 稳定性 |
|--------|-----------------|---------------|-----|
| 直接插入排序 | $O(n^2)$        | $O(1)$        | 稳定  |
| 起泡排序   | $O(n^2)$        | $O(1)$        | 稳定  |
| 快速排序   | $O(n \log_2 n)$ | $O(\log_2 n)$ | 不稳定 |
| 选择排序   | $O(n^2)$        | $O(1)$        | 稳定  |

注：上表"稳定性"及复杂度依据本汇编考点 75~78 的总结整理；源稿考点 79 原始对比图未能从文字版还原，如需精确对照请以原 PDF 为准。

## 整理说明

本页由《2026 广东专插本·计算机基础与程序设计·黄金知识汇编》PDF 文字稿整理而成，整理过程中已做以下处理与取舍：

- 1. **已清洗**：删除全部页码（如 - 1 - ）、## 第 N 页 分隔符、页脚推广文字（【优课必过】网校、zikao.p1tao.com、公众号【叶学长自考资料网】、专插本网课推广、开课微信等）；修复了 PDF 抽取产生的全角标点（如（ ），）、多余空白与断行，C 代码用代码块/行内代码包裹并尽量还原为可读形式（如 `int max (int x, int y)` → `int max(int x, int y)`）。
- 2. **无法完全还原的内容**（源 PDF 中为图片/示意图，文字版丢失）：
  - **考点58 双向链表的插入**：源稿仅有标题，未展开具体插入步骤。
  - **考点65 遍历二叉树**：算术表达式二叉树的树形结构图无法还原，仅保留先序/中序/后序/层序遍历结果。
  - **考点66 哈夫曼树构造**：四个权值（7、5、2、4）构造哈夫曼树的示意图无法还原，仅保留文字步骤与结论。
  - **考点79 排序算法比较**：源稿为对比图表，文字版未能还原原始对比内容，已依据前述考点 75~78 的总结做了近似归纳，精确对照请以原 PDF 为准。
  - 考点4 进制转换、考点53 矩阵相乘频度、考点73 哈希表探测、邻接矩阵等处的数学公式/求和符号在原 PDF 中为图片，已用文字（含  $\wedge$ 、 $\Sigma$ 、 $\lfloor \rfloor$  等近似表述）还原。



1. **忠实性**：仅整理源稿已有的知识，未发明源稿没有的内容；源稿中的例题与代码均保留并合并了分散在多页的同一考点内容。代码中的中文输出提示（如"input 10 numbers"）按源稿原样保留。
2. **覆盖范围**：C语言程序设计（第一章~第十章，考点 1~50）+ 数据结构（第一章~第八章，考点 51~79），共 18 章、79 个考点，无遗漏。